

# **Software Requirements Document**

**CineSeat**

**— Movie Theatre Booking System**

**Prepared by: Nunna Ravi Varma**

**Date: August 5, 2026**

# Table of contents:

- 1. Introduction**
- 2. Project Objectives**
- 3. System Overview**
- 4. Detailed Requirements**
- 5. Scope Definition**
- 6. Constraints**
- 7. Risks**
- 8. Milestones**

# 1. Introduction

## 1.1 Purpose

This Software Requirements Document (SRD) defines the objectives, functional and non-functional requirements, scope, constraints, risks, and milestones for CineSeat, a movie theatre seat booking system. It serves as the blueprint guiding design and implementation across a 6-week internship engagement.

## 1.2 Intended Audience

This document is intended for the developer (self), technical reviewers evaluating the portfolio project, and any collaborators who may extend the system in later phases.

## 1.3 Project Background

Post-pandemic, ticketing volume for cinemas and live events has grown substantially, and platforms like BookMyShow and Ticketmaster demonstrate sustained demand for reliable, high-concurrency seat reservation systems. CineSeat is modeled on these platforms as a way to demonstrate strong data structures and algorithms (DSA) knowledge alongside practical backend engineering in Java: concurrency control (via Java's concurrency utilities), caching, persistence, and REST API design.

## 1.4 Definitions and Acronyms

| Term             | Definition                                                                             |
|------------------|----------------------------------------------------------------------------------------|
| Hold             | A temporary, time-limited claim on a seat while a user completes payment.              |
| Sweep Thread     | A background process that expires stale holds using a min-heap ordered by expiry time. |
| VIP/Senior Queue | A max-heap based priority waitlist for premium or accessibility seating.               |
| SRD              | Software Requirements Document.                                                        |
| API              | Application Programming Interface.                                                     |

# 2. Project Objectives

The primary objectives of CineSeat are:

- Enable users to browse shows across multiple theaters and screens, and view real-time seat availability.
- Allow a user to reserve one or more seats, holding them for 10 minutes pending payment.
- Automatically release held seats that are not paid for within the hold window.
- Process payments and confirm reservations atomically with seat status updates.
- Notify users when a selected seat becomes unavailable due to a concurrent booking.

- Prioritize VIP and senior-citizen seat requests fairly when demand exceeds supply.
- Demonstrate correct handling of concurrent access to a shared, limited resource (seats).

## 3. System Overview

### 3.1 System Context

CineSeat consists of a backend service (Java, Spring Boot) exposing a booking API, backed by an in-memory data layer in early phases and a persistent PostgreSQL database with Redis caching in later phases. An optional React frontend renders a visual seat map in the final phase.

### 3.2 Core Entities

| Entity      | Description                                                                                      |
|-------------|--------------------------------------------------------------------------------------------------|
| Theater     | A physical location containing one or more screens.                                              |
| Screen      | A single auditorium with a fixed seat layout, modeled as a 2D grid.                              |
| Show        | A scheduled screening of a movie on a specific screen at a specific time.                        |
| Seat        | An individual seat with a status (available, held, booked) and category (standard, VIP, senior). |
| Reservation | A record linking a user, a show, and one or more seats, with a status and expiry.                |

### 3.3 DSA-to-Feature Mapping

| Feature                                       | Data Structure / Technique                                             |
|-----------------------------------------------|------------------------------------------------------------------------|
| Screen seat layout                            | 2D grid (Seat[][] or List<List<Seat>>)                                 |
| O(1) seat status lookup                       | Hashmap (HashMap / ConcurrentHashMap)                                  |
| Hold expiration (10-min timer)                | Min-heap (PriorityQueue) ordered by expiresAt, background sweep thread |
| VIP / senior priority queue                   | Max-heap (PriorityQueue with reverse comparator)                       |
| Finding N consecutive free seats              | Sliding-window scan over the seat grid                                 |
| Concurrent hold correctness (single instance) | Lock-based atomic check-and-set (ReentrantLock or synchronized)        |

### 3.4 Concurrency & Locking Strategy

Seat-hold correctness depends on where contention is resolved. This project distinguishes two deployment scales, since the correct locking mechanism differs between them.

### 3.4.1 Single-Instance Locking (Weeks 1–6 scope)

With one running application instance, all seat state lives in a single JVM heap. A per-seat `java.util.concurrent.locks.ReentrantLock` (or a synchronized block keyed by seat ID) is sufficient to guarantee that exactly one thread wins a hold attempt, as validated by the concurrent thread test in the prototype. This is the locking mechanism targeted for the internship deliverables.

### 3.4.2 Distributed Locking (documented upgrade path, out of internship scope)

If CineSeat is scaled horizontally across multiple Docker application instances behind a load balancer, in-memory Java locks stop providing mutual exclusion — each instance holds its own JVM heap, so two instances could each grant the same seat. The documented upgrade path is to replace local locks with a Redis-backed distributed lock via Redisson (RLock), which uses Redis's atomic operations to coordinate mutual exclusion across all instances. This is captured here as an architectural decision record for future scaling, not as a Week 1–6 build target.

### 3.4.3 Database Locking Strategy

When a hold is persisted to PostgreSQL, the system must choose between two locking strategies for the seat row:

| Strategy            | Mechanism                                                                                                                                 | Trade-off                                                                                                                                                                            |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pessimistic Locking | <code>SELECT ... FOR UPDATE</code> on the seat row during hold creation                                                                   | Guarantees serialized access at the database level, preventing lost updates under high contention; reduces throughput since competing transactions block until the lock is released. |
| Optimistic Locking  | A <code>@Version</code> column on the JPA seat/reservation entity; JPA rejects a stale write with an <code>OptimisticLockException</code> | Higher throughput under low-to-moderate contention since no row lock is held; requires explicit retry logic in the service layer when a conflict is detected.                        |

Decision: given that seat holds are a naturally high-contention operation (many users may target the same seat within the same second for popular shows), Pessimistic Locking (`SELECT ... FOR UPDATE`) is the chosen strategy for the hold-creation transaction.

## 4. Detailed Requirements

### 4.1 Functional Requirements

| ID    | Requirement                                                                          | Priority |
|-------|--------------------------------------------------------------------------------------|----------|
| FR-1  | Users can view a list of theaters, screens, and shows.                               | High     |
| FR-2  | Users can view real-time seat availability for a selected show.                      | High     |
| FR-3  | Users can select and hold one or more seats for 10 minutes.                          | High     |
| FR-4  | Held seats are automatically released if payment is not completed within 10 minutes. | High     |
| FR-5  | Users can complete payment to confirm a held reservation.                            | High     |
| FR-6  | System notifies a user if a seat they attempted to select was taken by another user. | High     |
| FR-7  | System supports VIP and senior-citizen priority queuing for high-demand shows.       | Medium   |
| FR-8  | Users can cancel a confirmed reservation before the show time.                       | Medium   |
| FR-9  | Admins can add/edit theaters, screens, shows, and seat pricing.                      | Medium   |
| FR-10 | System supports concurrent booking attempts without double-allocating a seat.        | High     |

### 4.2 Non-Functional Requirements

| ID     | Category                | Requirement                                                                                                           |
|--------|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| NFR-1  | Performance             | Seat status lookup must resolve in $O(1)$ average time.                                                               |
| NFR-2  | Performance             | Hold expiry checks must be $O(\log n)$ amortized via lazy heap deletion.                                              |
| NFR-3  | Concurrency             | Exactly one of $N$ concurrent requests for the same seat must succeed on a single instance; others must fail cleanly. |
| NFR-4  | Reliability             | No seat may be simultaneously held or booked by two different reservations.                                           |
| NFR-5  | Scalability             | Design must support scaling to multiple theaters and thousands of seats without architectural rework.                 |
| NFR-6  | Security                | Payment data must not be persisted in plaintext; API must validate all inputs.                                        |
| NFR-7  | Usability               | Seat availability must be visibly updated within 1 second of a status change.                                         |
| NFR-8  | Maintainability         | Codebase must be modular (data layer, business logic, API layer separated).                                           |
| NFR-9  | Portability             | Backend must build and run on Ubuntu/Debian using standard JDK and Maven/Gradle tooling.                              |
| NFR-10 | Distributed Scalability | Locking strategy must have a documented, non-breaking upgrade path from single-instance (ReentrantLock) to            |

| ID | Category | Requirement                                                                             |
|----|----------|-----------------------------------------------------------------------------------------|
|    |          | multi-instance (Redisson distributed lock) without changing the seat-hold API contract. |

## 5. Scope Definition

### 5.1 In Scope

- Seat inventory management via 2D grid + hashmap
- Hold/expire workflow with min-heap-based timers
- VIP/senior priority handling via max-heap
- Concurrency-safe seat locking (lock-based, using Java concurrency utilities)
- Payment confirmation (simulated or via a basic payment API integration)
- REST API layer for booking operations, built with Spring Boot
- PostgreSQL persistence (via JDBC/Spring Data JPA) and Redis caching (via Jedis/Lettuce)
- Containerized deployment via Docker

### 5.2 Out of Scope

- Multi-currency or international tax handling
- Loyalty/rewards programs
- Native mobile applications (web/API only)
- Third-party theater integrations beyond a single simulated payment gateway
- Multi-language localization

## 6. Constraints

- Implementation language is Java; this favors a mature ecosystem (Spring Boot, JUnit, JPA) for concurrency utilities, API development, and testing.
- Development is solo, following a 6-week internship task sequence (scoping → architecture → roadmap → testing strategy → code review/debugging → retrospective) — features are prioritized to fit each week's deliverable.
- Target platform is Ubuntu/Debian; JDK, Maven/Gradle, and container tooling installed via apt or SDKMAN.
- Payment processing will use a sandbox/test API rather than a live payment processor.
- No dedicated QA resource; testing relies on JUnit/Mockito and self-review.

## 7. Risks

| Risk                                                                  | Impact | Mitigation                                                                                                                                         |
|-----------------------------------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Race condition double-books a seat                                    | High   | Lock-based atomic check-and-set (ReentrantLock) validated via concurrent thread tests.                                                             |
| In-memory lock breaks under multi-instance scaling                    | Medium | Documented upgrade path to Redis-backed distributed locks (Redisson) before horizontal scaling; not required for single-instance internship scope. |
| Gap between in-memory check and durable DB write allows a lost update | Medium | Pessimistic locking (SELECT ... FOR UPDATE) on the seat row at persistence time closes the gap.                                                    |
| Hold expiry sweep falls behind under load                             | Medium | Lazy heap deletion keeps expiry checks $O(\log n)$ amortized.                                                                                      |
| Scope creep from optional features (frontend, extra payment methods)  | Medium | Strict week-by-week deliverable scope; optional items deferred to the future roadmap.                                                              |
| Underestimating integration time for DB/cache/API layers              | Medium | Buffer time built into Weeks 3–5; core DSA logic completed first.                                                                                  |
| Solo development limits code review depth                             | Low    | Rely on automated tests and structured self-review checklists.                                                                                     |

## 8. Milestones

This project is developed across the 6-week Junior Software Developer Intern program (31-Jul-2026 to 11-Sep-2026). Each internship week has a fixed documentation deliverable; the corresponding CineSeat technical build activity is noted alongside it.

| Week | Internship Deliverable                                           | CineSeat Technical Focus                                                                                                                                                            |
|------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Project Scoping & Software Requirements Document (this document) | Define entities, functional/non-functional requirements, DSA-to-feature mapping, scope.                                                                                             |
| 2    | System Architecture Design & Module Breakdown                    | Detailed architecture diagram; UI, business logic, data access, and integration module design; Java technology stack rationale.                                                     |
| 3    | Development Roadmap & Implementation Strategy                    | Stage-by-stage coding plan: seat grid, hold/expiry heap, VIP/senior heap, persistence, API — with milestones and resourcing.                                                        |
| 4    | Testing Strategy & QA Plan                                       | Unit/integration/system test design for concurrency correctness, hold expiry, and payment flow; JUnit/Mockito test cases.                                                           |
| 5    | Code Review Process & Debugging Strategy                         | Review checklist and debugging methodology for concurrency bugs, race conditions, and heap edge cases.                                                                              |
| 6    | Final Evaluation, Retrospective & Future Roadmap                 | Project retrospective against KPIs; roadmap for post-internship enhancements (e.g., React seat map, live payment gateway, Redisson distributed locking for multi-instance scaling). |